

```
#FINITE POTENTIAL WELL
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
from scipy.sparse import diags
```

```
from scipy.linalg import eigh
```

```
#Constants and grid
```

```
e = 1.602e-19; L = 1e-9; m = 9.109e-31; hbar = 1.055e-34
```

```
N, k = 400, 5
```

```
x = np.linspace(L/(N+1), L-L/(N+1), N); dx = x[1]-x[0]
```

```
# Hamiltonian
```

```
A = diags([1,-2,1], [-1,0,1], shape=(N,N)).toarray()
```

```
H = -(hbar**2)/(2*m*dx**2) * A
```

```
# Solve
```

```
evals, evects = eigh(H)
```

```
psi = evects[:, :k] / np.sqrt(np.trapz(evects[:, :k]**2, x, axis=0))
```

```
# Analytical energies
```

```
E_ana = (np.arange(1,k+1)**2 * np.pi**2 * hbar**2)/(2*m*L**2)/e
```

```
# Print table
```

```
print("\nFinite Square Well Energies (L = 1 nm)")
```

```
print("n Numerical (eV) Analytical (eV) Error")
```

```
for j in range(k):
```

```
    E_num = evals[j]/e
```

```
    print(f"{j+1:<2d} {E_num: .6e} {E_ana[j]: .6e} {E_num-E_ana[j]: .3e}")
```

```
#Small H example for N=6
```

```
N_small = 6; dx_s = L/(N_small+1)
```

```
A_s = np.diag(-2*np.ones(N_small)) + np.diag(np.ones(N_small-1),1) + np.diag(np.ones(N_small-1),-1)
```

```
H_s = -(hbar**2)/(2*m*dx_s**2) * A_s / e
```

```
print("\nExample Hamiltonian H (N=6) in eV:")
```

```
print(H_s)
```

```

# Figure 1: individual subplots
rows = (k+1)//2; cols = 2; fig1, axes = plt.subplots(rows, cols, figsize=(12,3*rows))
axes = axes.flatten()
for j in range(k):
    axes[j].plot(x, psi[:,j]**2)
    axes[j].set_title(f"n={j+1}"); axes[j].set_xlabel("x (m)"); axes[j].set_ylabel(" $\psi^2$ "); axes[j].grid(True)
for j in range(k, len(axes)): fig1.delaxes(axes[j])
fig1.suptitle("Finite Square Well — Individual Probability Densities", fontsize=16)
fig1.tight_layout(rect=[0,0,1,0.95])

```

```

#Figure 2: stacked offset plot
offset = psi.max()*1.2
plt.figure(figsize=(9,5))
for j in range(k):
    plt.plot(x, psi[:,j]**2 + j*offset, label=f"n={j+1}")
    plt.hlines(j*offset, x[0], x[-1], colors='gray', linewidth=0.3, alpha=0.4)
plt.xlabel("x (m)"); plt.ylabel(" $\psi^2(x) + \text{offset}$ ")
plt.title("Finite Square Well — Probability Densities (stacked)")
plt.grid(True); plt.legend(); plt.tight_layout(); plt.show()

```

```

#HYDROGEN

import numpy as np
import scipy.sparse as sp
import scipy.sparse.linalg as spla
import matplotlib.pyplot as plt

# ===== CONSTANTS =====
e2 = 14.3996
hbarc = 1973.0

```

```

m_ev = 0.511e6
h2_2m = hbarc**2 / (2*m_ev)

# ===== GRID =====
N = 4000
r_max = 200
dr = r_max / (N+1)
r = np.linspace(dr, r_max, N)

# ===== HAMILTONIAN =====
D2 = sp.diags([np.ones(N-1), -2*np.ones(N), np.ones(N-1)],
              offsets=[-1,0,1]) / dr**2
T = -h2_2m * D2
V = sp.diags(-e2 / r)

H = T + V

# ===== SOLVE =====
vals, vecs = spla.eigsh(H, k=5, which='SA')
idx = np.argsort(vals)
vals = vals[idx]
vecs = vecs[:, idx]

# ===== PRINT =====
print("Hydrogen energies:")
for n,E in enumerate(vals,1):
    print(f"n={n}: {E:.6f} eV")

# ===== 2x3 SUBPLOTS =====
fig, axs = plt.subplots(3,2,figsize=(14,12))
axs = axs.flatten()

for i in range(5):
    u = vecs[:,i].real
    u /= np.sqrt(np.trapz(u*u, r))
    axs[i].plot(r, u*u)

```

```

    axs[i].set_title(f"n = {i+1}")
    axs[i].set_xlabel("r (Å)")
    axs[i].set_ylabel("|u(r)|2")
    axs[i].grid(True)

```

```

axs[5].axis("off")
fig.suptitle("Hydrogen Probability Densities |u(r)|2")
plt.tight_layout(rect=[0,0,1,0.95])

```

```

# ===== STACKED OFFSET PLOT =====

```

```

plt.figure(figsize=(8,5))
for i in range(5):
    u = vecs[:,i].real
    u /= np.sqrt(np.trapz(u*u, r))
    plt.plot(r, u*u + i*0.02, label=f"n={i+1}")

```

```

plt.xlim(0,40)
plt.xlabel("r (Å)")
plt.ylabel("|u(r)|2 + offset")
plt.title("Probability Density |u(r)|2")
plt.grid(True)
plt.legend()
plt.tight_layout()
plt.show()

```

```

#HARMONIC AND ANHARMONIC

```

```

import numpy as np
import matplotlib.pyplot as plt
from scipy.sparse import diags
from scipy.linalg import eigh

```

```

# --- Grid & constants ---

hbar_c, m, k = 197.3, 940.0, 100.0

xmin, xmax, n_points = -5, 5, 1000

x = np.linspace(xmin, xmax, n_points)

dx = x[1]-x[0]

# --- User input ---

b = float(input("Enter b (MeV/fm³), e.g. 0, 10, 30: "))

# --- Kinetic energy (finite difference) ---

pref = -(hbar_c**2)/(2*m*dx**2)

T = diags([1,-2,1], [-1,0,1], shape=(n_points,n_points)).toarray() * pref

# --- Potential & Hamiltonian ---

V_diag = 0.5*k*x**2 + b/3*x**3

H = T + np.diag(V_diag)

# --- Solve ---

evals, evects = eigh(H)

k_levels = 5

psi = evects[:, :k_levels] / np.sqrt(np.trapz(evects[:, :k_levels]**2, x, axis=0))

# --- Print energies ---

print(f"\nResults for b = {b} MeV/fm³:")

for i,E in enumerate(evals[:k_levels],1):

    print(f"  Energy level {i}: {E:.3f} MeV")

# --- Figure 1: subplots ---

fig1, axes = plt.subplots((k_levels+1)//2, 2, figsize=(12,3*((k_levels+1)//2)))

axes = axes.flatten()

for i in range(k_levels):

    axes[i].plot(x, psi[:,i], label=f"ψ_{i+1}")

    axes[i].set_title(f"Level {i+1} = {evals[i]:.3f} MeV"); axes[i].grid(True); axes[i].legend()

for j in range(k_levels,len(axes)): fig1.delaxes(axes[j])

fig1.suptitle(f"Wavefunctions for Anharmonic Oscillator (b={b})", fontsize=16)

```

```
fig1.tight_layout(rect=[0,0,1,0.95])
```

```
# --- Figure 2: overlay ---
```

```
fig2, ax2 = plt.subplots(figsize=(8,6))
```

```
colors = plt.cm.viridis(np.linspace(0,1,k_levels))
```

```
for i in range(k_levels):
```

```
    ax2.plot(x, psi[:,i]/np.max(np.abs(psi[:,i])), color=colors[i], label=f"n={i+1}")
```

```
ax2.plot(x, V_diag/np.max(np.abs(V_diag)), 'k--', label='Scaled potential')
```

```
ax2.set_title(f"Overlay of first {k_levels} wavefunctions (b={b})")
```

```
ax2.set_xlabel("x (fm)"); ax2.set_ylabel(" $\psi_n(x)$  (scaled)"); ax2.grid(True); ax2.legend()
```

```
plt.show()
```

```
#SPHERICAL HARMONICS
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
from scipy.special import sph_harm
```

```
# --- Spherical to Cartesian ---
```

```
def sph_to_cart(r, th, ph):
```

```
    return (r*np.sin(th)*np.cos(ph),
```

```
            r*np.sin(th)*np.sin(ph),
```

```
            r*np.cos(th))
```

```
# --- Grid ---
```

```
theta = np.linspace(0, np.pi, 150)
```

```
phi = np.linspace(0, 2*np.pi, 150)
```

```
TH, PH = np.meshgrid(theta, phi)
```

```
# --- (l,m) pairs to plot ---
```

```
pairs = [(0,0),(1,0),(1,1),(2,0),(2,1),(2,2),(3,0),(3,1),(3,2),(3,3)]
```

```
plots_per_fig = 4
```

```

nfigs = (len(pairs) + plots_per_fig - 1) // plots_per_fig
for f in range(nfigs):
    fig = plt.figure(figsize=(12,12))

    for i in range(plots_per_fig):
        idx = f*plots_per_fig + i
        if idx >= len(pairs): break

        l, m = pairs[idx]

        Y = sph_harm(m, l, PH, TH)
        P = (Y.real**2 + Y.imag**2)
        P /= P.max()

        r = 1 + 0.5*P
        X, Yc, Z = sph_to_cart(r, TH, PH)

        ax = fig.add_subplot(2,2,i+1, projection='3d')
        ax.plot_surface(X, Yc, Z, facecolors=plt.cm.viridis(P), rstride=1, cstride=1, linewidth=0)
        ax.set_axis_off()
        ax.set_box_aspect([1,1,1])
        ax.set_title(f"|Y_{{{l}}, {m}}|^2")

    plt.tight_layout()
plt.show()
print("All selected spherical harmonics plotted successfully!")

```